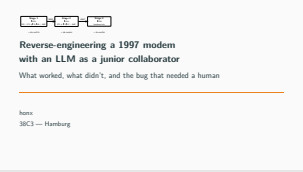
Reverse-engineering a 1997 modem with an LLM as a junior collaborator

What worked, what didn't, and the bug that needed a human

honx

38C3 — Hamburg



Welcome.

I'm going to talk about what happened when I let Claude — an LLM — loose on a real reverse-engineering problem. Specifically: build a Ghidra processor module for a chip that Ghidra has never heard of, then use it to take apart a 1997 modem firmware.

Two reasons this might be interesting at Congress, especially this year:

(1) The CCC audience is, correctly, sceptical of AI hype. I don't have any hype to sell you. I have a four-day case study with both successes and a substantive bug that needed a human to catch.

(2) The artefact is real. It's checked in, it has tests, and you can download it tonight and feed your own L3902 firmware to it.

Talk is roughly 45 minutes. I'll save questions for the end.

Disclaimers

Disclaimers

What this talk is *not*

- "AI replaces reverse engineers."
- "You can prompt your way to a working tool."
- "LLMs are about to make Ghidra obsolete."
- Vendor pitch. There is no product here. Just a git repo.

Set expectations early. CCC audience is allergic to AI hype, with good reason. I want to acknowledge that up front.

I'm not selling anything. The only thing I'm advocating for is: if you're a reverse engineer and you're sceptical, don't dismiss this without trying it. Form your scepticism from first-hand experience.

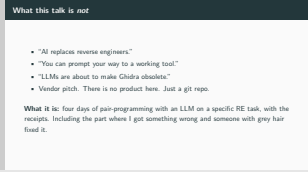
If you ARE running on hype, this talk should also slow you down. There is a real failure mode in the middle of the story.

The "grey hair" thing is the German custom of trusting older technical expertise — this is for the German contingent. Mention it lightly.

What this talk is *not*

- "AI replaces reverse engineers."
- "You can prompt your way to a working tool."
- "LLMs are about to make Ghidra obsolete."
- Vendor pitch. There is no product here. Just a git repo.

What it is: four days of pair-programming with an LLM on a specific RE task, with the receipts. Including the part where I got something wrong and someone with grey hair fixed it.



Set expectations early. CCC audience is allergic to AI hype, with good reason. I want to acknowledge that up front.

I'm not selling anything. The only thing I'm advocating for is: if you're a reverse engineer and you're sceptical, don't dismiss this without trying it. Form your scepticism from first-hand experience.

If you ARE running on hype, this talk should also slow you down. There is a real failure mode in the middle of the story.

The "grey hair" thing is the German custom of trusting older technical expertise — this is for the German contingent. Mention it lightly.

The deal

- Audience: you, sceptical.
- Subject: AI-assisted RE, on a real chip with real datasheets.
- Outcome metric: a working Ghidra module, a tested SLEIGH spec, and a 350-line analysis writeup. Or not. We'll see.
- Counterweight: the bug that proves the AI didn't really understand what it was doing.

The deal
- Audience: you, sceptical.
- Subject: AI-assisted RE, on a real chip with real datasheets.
- Outcome metric: a working Ghidra module, a tested SLEIGH spec, and a 350-line analysis writeup. Or not. We'll see.
- Counterweight: the bug that proves the AI didn't really understand what it was doing.

Frame the rest of the talk: this is a single concrete case study. Generalise from it cautiously, but it's at least one data point.

Audience may push back at the end. That's fine. I have specific things I'll say worked and specific things I'll say didn't, and the receipts are in the public repo.

The artefact

The artefact

A piece of 1997 hardware

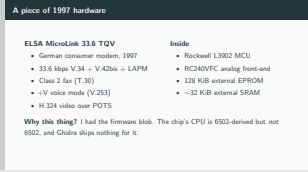
ELSA MicroLink 33.6 TQV

- German consumer modem, 1997
- 33.6 kbps V.34 + V.42bis + LAPM
- Class 2 fax (T.30)
- +V voice mode (V.253)
- H.324 video over POTS

Inside

- Rockwell L3902 MCU
- RC240VFC analog front-end
- 128 KiB external EPROM
- ∼32 KiB external SRAM

Why this thing? I had the firmware blob. The chip's CPU is 6502-derived but *not* 6502, and Ghidra ships nothing for it.



Frame the artefact briefly. The audience here mostly grew up with modems — this is nostalgia hardware for them. That's fine.

Key non-obvious detail: Class 2 fax + voice + H.324. This isn't a basic modem; it's a full V.253 voice modem, capable of fax sending/receiving and video conferencing over POTS. That's a non-trivial firmware budget in 128 KiB.

The "I had the firmware blob" — the audience will assume I dumped it or downloaded it. Either way, neither shipping it nor asking how I got it is the point. The point is the chip, which has zero open tooling.

The CPU: Rockwell R65C19

Enhanced 6502, late 1980s.

On top of stock 65C02:

- 12 new bit-manip / branch ops
- 21 new arithmetic ops (`MPY`, `MPA`, `NEG`, `ASR`, `ADD`, ...)
- 10 threaded-code ops (`NXT`, `TIP`, `JPI`, ...)
- 16-bit `W` reg (multiply accumulator)
- 16-bit `I` reg (Forth-style instruction pointer)

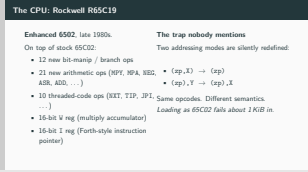
The trap nobody mentions

Two addressing modes are silently redefined:

- `(zp,X)` → `(zp)`
- `(zp),Y` → `(zp),X`

Same opcodes. Different semantics.

Loading as 65C02 fails about 1 KiB in.



The R65C19 is interesting because it's not "6502 with extra ops on the side". The new ops use opcodes that 6502 NMOS marked illegal — fine. But Rockwell also ALTERED two of the standard 6502 indirect modes.

The trap: if you load this firmware as 65C02, the disassembly looks correct for the first few hundred bytes (no indirect ops), then silently goes wrong. Everywhere there's a '(zp,X)' it should be a '(zp)'. Branch targets are wrong, control flow is wrong, and you're debugging a phantom.

Fixing this requires writing a SLEIGH processor spec from scratch. Which is what this talk is about.

Banking

The L3902 talks to its 128 KiB EPROM through 8 bank-select registers at $0018–$001F. Each one maps a single 8 KiB physical bank into one of eight 8 KiB windows in the CPU's 64 KiB view.

```
BSRn = | ES3 | ES2 | ES1 | ES0 | A16 | A15 | A14 | A13 |
         bit7  bit6  bit5  bit4  bit3  bit2  bit1  bit0
```

- A13–A16: 4 bits = 16 physical banks × 8 KiB = 128 KiB
- ES0–ES3: chip-select assertions (signal type, not address)

Hold this slide in your head. I will get this part wrong and have to come back to it.

This slide is foreshadowing. The BSR encoding turns out to be the exact thing the AI got wrong, and we'll come back to it in detail. I want the audience to see the encoding now so the later "I had it backwards" lands harder.

The four address bits cover all of the EPROM. The four ES bits do NOT contribute to the address — they're chip-select signals. This is something a human RE engineer who's worked with banked memory before would know instantly. The AI confidently invented a different model.

The plan

The plan

What I asked the LLM to do

Roughly:

1. Read the R65C19 datasheet. Build a SLEIGH spec covering the whole ISA, including the indirect-mode trap.
2. Read the C40/L39 manual. Build the L3902-specific bits: peripheral symbols, vector table, memory map.
3. Write a Ghidra loader that handles >64 KiB banked images.
4. Write tests: SLEIGH compile, opcode-decode fixture, end-to-end firmware import.
5. Run it. Show me what the firmware does.

This is not “one prompt.” Over four days I sent maybe 30 messages.

What I asked the LLM to do
Roughly:
1. Read the R65C19 datasheet. Build a SLEIGH spec covering the whole ISA, including the indirect-mode trap.
2. Read the C40/L39 manual. Build the L3902-specific bits: peripheral symbols, vector table, memory map.
3. Write a Ghidra loader that handles >64 KiB banked images.
4. Write tests: SLEIGH compile, opcode-decode fixture, end-to-end firmware import.
5. Run it. Show me what the firmware does.
This is not “one prompt.” Over four days I sent maybe 30 messages.

The framing matters. I didn’t open ChatGPT and ask "build a Ghidra module for a Rockwell L3902". I structured the work as a sequence of discrete tasks, each one verifiable.

Total interaction was maybe 30 messages over four sessions. Each session ended with something runnable. That’s the rhythm: AI does boilerplate, you check the output, fix what’s wrong, move on.

If you treat this as autonomous, you get plausible nonsense. If you treat it as pair-programming with someone who works very fast and has read every reference doc but has no judgement, you get a lot done.

What the LLM had access to

- Three vendor PDFs (R65C19 data sheet, C40/L39 TRM, RC240VFC)
- The 128 KiB firmware blob
- A working Ghidra install at `/home/honx/ghidra`
- Shell, file editing, the SLEIGH compiler, headless analysis
- No internet. No prior code. No examples.

The session was Claude Code (Anthropic's CLI agent). Same pattern should work in any agentic setup with shell + file editing.

What the LLM had access to
- Three vendor PDFs (R65C19 data sheet, C40/L39 TRM, RC240VFC)
- The 128 KiB firmware blob
- A working Ghidra install at /home/honx/ghidra
- Shell, file editing, the SLEIGH compiler, headless analysis
- No internet. No prior code. No examples.
The session was Claude Code (Anthropic's CLI agent). Same pattern should work in any agentic setup with shell + file editing.

Important: this is local. The LLM had file/shell access but no web access. It read the vendor PDFs through Anthropic's PDF tool, ran the SLEIGH compiler on actual files, and ran Ghidra headless.

Why it matters: the audience may assume "the AI just looked up existing R65C19 SLEIGH code online." It did not. There isn't any. The spec was derived from reading 46-page R65C19 data sheet OCR, including a 256-cell opcode matrix.

I'm using Claude Code specifically because it has good tool-use, but the workflow generalises. Cursor, Aider, anything with shell + file tools.

Building the SLEIGH spec

Building the SLEIGH spec

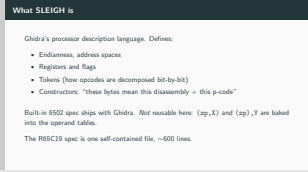
What SLEIGH is

Ghidra's processor description language. Defines:

- Endianness, address spaces
- Registers and flags
- Tokens (how opcodes are decomposed bit-by-bit)
- Constructors: "these bytes mean this disassembly + this p-code"

Built-in 6502 spec ships with Ghidra. *Not* reusable here: `(zp,X)` and `(zp),Y` are baked into the operand tables.

The R65C19 spec is one self-contained file, ∼600 lines.



Quick orientation for people who haven't written SLEIGH. It's a small domain-specific language for describing instruction sets. Pretty declarative. Reasonably well-suited to LLM work because it's mechanical: "this byte pattern means this mnemonic with these operands."

Why I didn't @include the existing 6502 spec: the addressing-mode remap is in the operand-resolution tables, which the 6502 spec defines globally. Overriding them is messier than rewriting the relevant constructors.

600 lines is small for an ISA spec. The bulk is the instruction constructors; tokens and macros are maybe 80 lines.

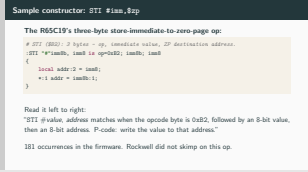
Sample constructor: `STI #imm,$zp`

The R65C19's three-byte store-immediate-to-zero-page op:

```
# STI ($B2): 3 bytes - op, immediate value, ZP destination address.
:STI "#"imm8b, imm8 is op=0xB2; imm8b; imm8
{
    local addr:2 = imm8;
    *:1 addr = imm8b:1;
}
```

Read it left to right:
"STI #*value*, *address* matches when the opcode byte is 0xB2, followed by an 8-bit value, then an 8-bit address. P-code: write the value to that address."

181 occurrences in the firmware. Rockwell did not skimp on this op.



Show one constructor in detail so the audience sees what SLEIGH looks like. Pick STI because it's the most-used R65C19 extension.
Bit of pedagogy: SLEIGH constructors have three parts. The display format string defines what gets printed in the listing. The bit-pattern match defines when this constructor fires. The p-code semantics block defines what the instruction actually does.
The token "imm8b" exists separately from "imm8" because SLEIGH won't let two operands of the same token type appear in one instruction. So the spec defines a parallel one-byte token for the second operand. That's a minor SLEIGH-grammar gotcha that the LLM did stumble on once and had to fix. Mention it briefly.

Indirect-mode remap

The thing 65C02 gets silently wrong on R65C19:

```
# OP1 = operand class for cc=01 ALU instructions.
#   bbb=0  (zp)         R65C19 zero-page indirect (was (zp,X) on 6502)
#   bbb=4  (zp),X       R65C19 zp indirect, post-indexed by X (was (zp),Y)
OP1: (imm8)     is bbb=0; imm8        { addr:2 = imm8; tmp:2 = *:2 addr;
                                        export *:1 tmp; }
OP1: (imm8),X   is bbb=4 & X; imm8    { addr:2 = imm8; tmp:2 = *:2 addr;
                                        tmp = tmp + zext(X); export *:1 tmp; }
```

Two slots in the operand table that the LLM had to flip from the stock 6502 versions. Without this, every `LDA ($xx),X` in the firmware decodes as `LDA ($xx),Y` — silently wrong.

This is the slot-flip that was the whole reason to write a custom spec. Show it.

This is the kind of thing where the LLM did well: it had read both the R65C19 datasheet's "instructions changed" table and the stock 6502 spec, and it correctly identified that the only way to express the remap was to redefine OP1's bbb=0 and bbb=4 entries.

Mechanical, well-specified, the kind of work LLMs are good at.

The opcode matrix

The R65C19 datasheet has a 16×16 opcode matrix. 256 cells, each labelled with mnemonic + addressing mode + cycle count.

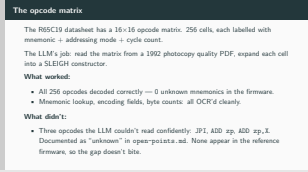
The LLM's job: read the matrix from a 1992 photocopy quality PDF, expand each cell into a SLEIGH constructor.

What worked:

- All 256 opcodes decoded correctly — 0 unknown mnemonics in the firmware.
- Mnemonic lookup, encoding fields, byte counts: all OCR'd cleanly.

What didn't:

- Three opcodes the LLM couldn't read confidently: `JPI`, `ADD zp`, `ADD zp,X`. Documented as "unknown" in `open-points.md`. None appear in the reference firmware, so the gap doesn't bite.



Be honest about the OCR step. It worked, but not perfectly. The LLM was conservative: when it couldn't read a cell with confidence, it flagged it and kept going.
That's good behaviour. The alternative — guessing — would have produced code that disassembled wrongly. Having it be honest about the OCR confidence level was important.
The three missing opcodes are real gaps but turn out not to matter for this firmware. We verified that with a static scan after the fact. If a future user runs the spec on a R65C19 image that uses JPI, they get an "unknown opcode" mark and can extend the spec.

The loader and the test suite

The loader and the test suite

Loader: banked memory

Java loader, ∼150 lines. Recognises an L3902 firmware image by size constraint (multiple of 8 KiB up to 512 KiB) and:

- Maps the first 64 KiB at `$0000–$FFFF` as ROM
- Layers uninitialised blocks for the on-chip I/O, RAM pages, CRC
- Reads the reset vector at `$FFFE` and creates a `_reset` function there
- For images >64 KiB, drops each additional 64 KiB chunk in as a `BANK_HIGH_n` overlay block

Bank-aware cross-references — "after this `STI #imm,$1B`, `$6200` reads from `file:0x0200`" — are **not** implemented.
That's listed under open points.

Loader: banked memory
Java loader, ∼150 lines. Recognises an L3902 firmware image by size constraint (multiple of 8 KiB up to 512 KiB) and:
- Maps the first 64 KiB at $0000–$FFFF as ROM
- Layers uninitialised blocks for the on-chip I/O, RAM pages, CRC
- Reads the reset vector at $FFFE and creates a _reset function there
- For images >64 KiB, drops each additional 64 KiB chunk in as a BANK_HIGH_n overlay block
Bank-aware cross-references — "after this STI #imm,$1B, $6200 reads from file:0x0200" — are not implemented.
That's listed under open points.

The loader is the least interesting part of the work. It's boilerplate Java that walks the bytes and creates Ghidra memory blocks. LLM did this well because there are 50 example loaders in Ghidra's source tree and the API is stable.

Caveat: the bank-aware cross-referencing IS the hard part, and it's NOT implemented. Today, when Ghidra encounters an STI immediate followed by a JMP to a bank-mapped address, it disassembles both correctly but doesn't model the bank switch — so it follows the JMP into whatever bytes were there before the switch.

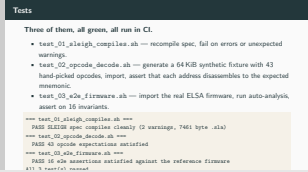
Building a bank-aware analyser would be a serious piece of work. It's the kind of thing where an LLM gets you 80finishes the engineering. It's on the open-points list.

Tests

Three of them, all green, all run in CI.

- `test_01_sleigh_compiles.sh` — recompile spec, fail on errors or unexpected warnings.
- `test_02_opcode_decode.sh` — generate a 64 KiB synthetic fixture with 43 hand-picked opcodes, import, assert that each address disassembles to the expected mnemonic.
- `test_03_e2e_firmware.sh` — import the real ELSA firmware, run auto-analysis, assert on 16 invariants.

```
=== test_01_sleigh_compiles.sh ===
  PASS SLEIGH spec compiles cleanly (2 warnings, 7461 byte .sla)
=== test_02_opcode_decode.sh ===
  PASS 43 opcode expectations satisfied
=== test_03_e2e_firmware.sh ===
  PASS 16 e2e assertions satisfied against the reference firmware
All 3 test(s) passed
```

This is a part of the workflow that I think is underappreciated. LLMs write tests well — if you ask them to. They will happily not write any tests if you don't.

The three tests cover three failure modes. Spec broken: test one catches it. One opcode misencoded: test two catches it. Real-world disassembly drifting: test three catches it.

The opcode-decode test surfaced two real bugs while I was writing it: the unconditional branch in the fixture meant later opcodes weren't linearly reachable (had to disassemble each address explicitly), and the bitindex display format wasn't what I'd guessed it would be. Tests catching test bugs early — fine, that's what tests are for.

The analysis

The analysis

First decode

Reset vector: file 0xFFFE/F = c0 ff → reset target $FFC0.

Disassembly at $FFC0:

```
$FFC0  78          SEI                ; mask all IRQs
$FFC1  B2 70 1B    STI #$70, $1B      ; BSR3 := $70 - bank-switch
$FFC4  4C 00 62    JMP $6200          ; jump into newly-mapped bank
```

This is the moment the spec "works."

- Three instructions disassembled cleanly.
- One of them is the R65C19 STI extension that no stock 6502 spec can decode.
- Auto-analysis finds 204 functions in 4 seconds.

This is the demo moment. The first three instructions of the firmware include the R65C19 extension we wrote SLEIGH for. If the spec were broken, the disassembly would be visibly wrong here.

204 functions in 4 seconds is Ghidra's auto-analysis. That's not "the LLM understood the firmware" — that's "Ghidra's analyser was able to follow flow and identify subroutines, given a working processor module."

For a sceptical audience, this is where the talk says "the spec demonstrably works on real input."

The rest of the talk is what we USE the working spec to learn.

What the firmware does, by string evidence

The image carries its own functional spec, in printable strings. The LLM dumped them, grouped them, and wrote them up.

`(C) ROCKWELL / ELSA Aachen` branding (1997-07-18 build)

`V42bis, TxSymRa Bd, TxPrecoding, V34PwrRed...` V.34 / V.42bis stats labels

`ROCKWELL;ADPCM;16` non-standard voice-mode codec

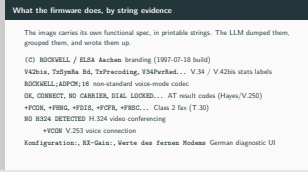
`OK, CONNECT, NO CARRIER, DIAL LOCKED...` AT result codes (Hayes/V.250)

`+FCON, +FHNG, +FDIS, +FCFR, +FNSC...` Class 2 fax (T.30)

`NO H324 DETECTED` H.324 video conferencing

`+VCON` V.253 voice connection

`Konfiguration:, RX-Gain:, Werte des fernen Modems` German diagnostic UI



Strings are the cheap part of RE — they tell you, for free, what the firmware claims to do. The LLM categorised these from a flat list of 600+ strings.

This is honest pattern-matching. The LLM didn't "understand" what T.30 is — it labelled the +F* strings as fax response codes because that's what they are in the standard, and an LLM has read the standard. That's appropriate use of the model's prior.

Important call-out: the same firmware blob services three SKUs (14.4TQ, 28.8TQ, 33.6TQ). Single image, three product configurations keyed off some flag.

Architecture, by code

Trampoline-driven dispatch.

204 functions, 2179 instructions in the boot-time view. Many follow a 6-byte template:

```
LDY $85
RND
STI #$03, $41           ; command opcode
STI #$12, $40           ; sub-command
JSR $E81B               ; central dispatcher
RTS
```

- One stub per AT command. ($41,$40) = command pair.
- Dispatcher at $E81B does the table lookup once, for everyone.
- STI alone accounts for 8.3% of the instruction stream.

This is the architectural insight: the firmware is built around a single dispatcher with dozens of tiny per-command stubs. Classic Rockwell modem-firmware style — the dispatch table is in ROM, and everything else is parameter setup.

8.3would be 0the third-most-common instruction.

Bit-manipulation density (BAR/BAS/SBA/RBA/BBR/BBS/SMB/RMB) is 8.6That's also unusual. The firmware spends a lot of time setting and reading individual bits in control registers — which makes sense for something whose job is talking to peripherals.

The bug

The bug

The bug.

Where the LLM's confident wrongness met a human reviewer.

The bug.
Where the LLM's confident wrongness met a human reviewer.

This is the part I wanted to do justice to. The talk would be cheap if I just showed everything that worked. The real lesson is in the place the AI got something wrong, sounded confident about it, and a human caught it.

Pause here. Slow down. The next four slides are the most important ones in the talk for a sceptical audience.

What the LLM said

After analysing the boot stub the LLM wrote, in the analysis document:

> *"The 128 KiB image is laid out so that **the first 64 KiB is what the L3902 sees while only ES0 is active**, and the second 64 KiB is what it sees once ES1, ES2 and ES3 are also asserted. . . . The boot stub bank-switches BSR3 to expose chip-2 in the $6000-$7FFF window, then jumps to $6200."*

This is the "two physical chips" theory. It is plausible, parses the BSR write, sounds technically literate.

It is also wrong.

What the LLM said
After analysing the boot stub the LLM wrote, in the analysis document:
"The 128 KiB image is laid out so that the first 64 KiB is what the L3902 sees while only ES0 is active, and the second 64 KiB is what it sees once ES1, ES2 and ES3 are also asserted. . . . The boot stub bank-switches BSR3 to expose chip-2 in the $6000-$7FFF window, then jumps to $6200."
This is the "two physical chips" theory. It is plausible, parses the BSR write, sounds technically literate.
It is also wrong.

Show the actual quote from the analysis writeup as it stood after the LLM's first pass. This is what got committed to the repo before review.

Why it sounds plausible: it parses the BSR encoding, it explains where the second 64 KiB comes from, it accounts for the chip-select bits. It's an internally-consistent model.

Why it's wrong: it ignores the part of the BSR encoding that says A13-A16 is FOUR bits, not three. Four bits = 16 banks. 16 banks of 8 KiB = 128 KiB. The whole EPROM, addressed by A13-A16 alone.

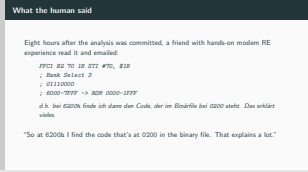
The LLM read the BSR encoding correctly, then constructed a system model that didn't match the encoding it had just read. That's a specific failure mode worth naming: confident pattern-fitting without checking.

What the human said

Eight hours after the analysis was committed, a friend with hands-on modem RE experience read it and emailed:

> *FFC1 B2 70 1B STI #70, $1B*
> *; Bank Select 3*
> *; 01110000*
> *; 6000-7FFF -> ROM 0000-1FFF*
>
> *d.h. bei 6200h finde ich dann den Code, der im Binärfile bei 0200 steht. Das erklärt vieles.*

"So at 6200h I find the code that's at 0200 in the binary file. That explains a lot."



Read the friend's email. Slow down. This is what good RE review looks like.

He didn't say "you're wrong". He said "here's the encoding, here's what it means, here's where the code actually is". One line.

The encoding-decoding is unambiguous: BSR3 equals hex seventy, in binary that's zero-one-one-one zero-zero-zero-zero. The four low bits, which are A13 through A16, are all zero — meaning physical bank zero. Physical bank zero of a 128 KiB EPROM is file offsets zero through 0x1FFF. So logical 6200 hex maps to file 0x0200, period.

The LLM's two-chips model said 6200 hex maps to file 0x10200. The reviewer checked. It doesn't.

What was actually at file 0x0200

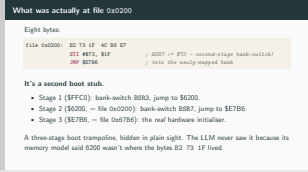
Eight bytes:

```
file 0x0200:  B2 73 1F  4C B6 E7
              STI #$73, $1F          ; BSR7 := $73 - second-stage bank-switch!
              JMP $E7B6              ; into the newly-mapped bank
```

It's a second boot stub.

- Stage 1 ($FFC0): bank-switch BSR3, jump to $6200.
- Stage 2 ($6200, = file 0x0200): bank-switch BSR7, jump to $E7B6.
- Stage 3 ($E7B6, = file 0x67B6): the *real* hardware initialiser.

A three-stage boot trampoline, hidden in plain sight. The LLM never saw it because its memory model said 6200 wasn't where the bytes B2 73 1F lived.



Walk through the chain. This is the key technical content.

When you do follow the chain correctly, you discover the firmware has a beautifully economical three-stage boot. Each stage is two instructions before its 'JMP'. Each stage bank-switches its successor into view, then jumps. The third stage finally programs all eight BSRs to the runtime memory map.

This is GOOD firmware engineering, the kind you only see when every byte of ROM costs money. It's also the kind of structural insight the LLM had no chance of finding because its starting model was wrong.

This was not a small bug. It changed our entire reading of the firmware. Before the correction: "first 64 KiB is data, second 64 KiB is code." After: "single 128 KiB image, three-stage trampoline, all the code lives in low banks."

The lesson

The LLM had read:

- The full BSR encoding.
- The 128 KiB file size.
- The L39 TRM's "up to 512 KiB external memory" claim.

It had *enough information* to derive the right model.
It instead constructed a confident, plausible, wrong one.

Specific failure mode: pattern-fit a coherent narrative around incomplete reasoning, present it as established fact.

Specific corrective: a human with hands-on experience reading the encoding from the bit pattern and saying "no, that's bank zero, the file offset is 0x0200."

The bit pattern was visible to both of us. Only one of us read it correctly.

The lesson

The LLM had read:

- The full BSR encoding.
- The 128 KiB file size.
- The L39 TRM's "up to 512 KiB external memory" claim.

It had *enough information* to derive the right model.
It instead constructed a confident, plausible, wrong one.

Specific failure mode: pattern-fit a coherent narrative around incomplete reasoning, present it as established fact.

Specific corrective: a human with hands-on experience reading the encoding from the bit pattern and saying "no, that's bank zero, the file offset is 0x0200."

The bit pattern was visible to both of us. Only one of us read it correctly.

Be precise about what the failure was. Not "the LLM hallucinated". Not "the LLM didn't have enough information". The information was all there. The LLM constructed a plausible-but-wrong model and wrote it up as fact.

This is the failure mode that every serious AI user has to learn to recognise. The LLM's outputs are calibrated for plausibility, not correctness. When the two diverge, plausibility wins, until a human intervenes.

The corrective is not "trust the human more". The corrective is to have the LLM and the human reading the same artefact, and to send the LLM back to the artefact with a specific question whenever something doesn't add up.

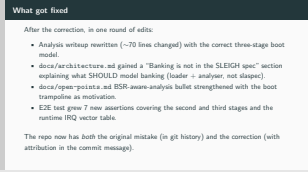
In this case the friend didn't even need to know the L3902 to be right. He just READ THE ENCODING. The LLM had read it too. The difference is the friend then USED what he'd read to derive the answer. The LLM had moved on.

What got fixed

After the correction, in one round of edits:

- Analysis writeup rewritten (∼70 lines changed) with the correct three-stage boot model.
- `docs/architecture.md` gained a "Banking is not in the SLEIGH spec" section explaining what SHOULD model banking (loader + analyser, not slaspec).
- `docs/open-points.md` BSR-aware-analysis bullet strengthened with the boot trampoline as motivation.
- E2E test grew 7 new assertions covering the second and third stages and the runtime IRQ vector table.

The repo now has *both* the original mistake (in git history) and the correction (with attribution in the commit message).



Show what shipped after the correction. The point is that the correction was small, surgical, and produced a better artefact.

The git history matters. Both commits are in the repo. Anyone who clones it can see the wrong version, the email-driven correction, and the new tests that prevent the same kind of mistake from recurring silently.

This is what "AI in the loop" should look like in practice. Not "AI replaces human review". "AI does work fast, human reviews, mistakes get pushed back, repo gets better."

Honest assessment

What worked well

- **SLEIGH grammar and constructors** — mechanical, well-specified, large existing corpus. LLM strength.
- **Loader boilerplate** — 50 example loaders in Ghidra source. LLM had absorbed the API.
- **Test scaffolding** — shell drivers, headless scripts, fixture generation. Mostly boilerplate.
- **Documentation at multiple levels** — overview, usage, architecture, open-points. Tedious for humans, fast for an LLM.
- **Refactoring under spec** — "move this to a separate cspec file," "rename `I` to `Iflag` to avoid the threaded-code reg collision". Mechanical.

What worked well
- **SLEIGH grammar and constructors** — mechanical, well-specified, large existing corpus. LLM strength.
- **Loader boilerplate** — 50 example loaders in Ghidra source. LLM had absorbed the API.
- **Test scaffolding** — shell drivers, headless scripts, fixture generation. Mostly boilerplate.
- **Documentation at multiple levels** — overview, usage, architecture, open-points. Tedious for humans, fast for an LLM.
- **Refactoring under spec** — "move this to a separate cspec file," "rename I to Iflag to avoid the threaded-code reg collision". Mechanical.

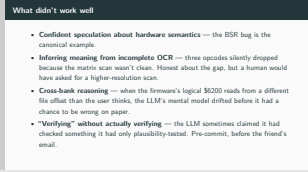
Be specific about what the LLM contributed real value to. These are all tasks where the LLM's training has seen many similar examples, and the work is mostly applying known patterns to a new instance.

Documentation is underrated. I now have FOUR documents covering this project at different levels, plus a 350-line firmware analysis. I would not have written that documentation by hand. Not in four days, maybe not ever.

Tests too. Without prompting, the LLM defaulted to "no tests". With explicit asks for tests, it produced a respectable suite. That's a prompting issue, not a capability issue.

What didn't work well

- **Confident speculation about hardware semantics** — the BSR bug is the canonical example.
- **Inferring meaning from incomplete OCR** — three opcodes silently dropped because the matrix scan wasn't clean. Honest about the gap, but a human would have asked for a higher-resolution scan.
- **Cross-bank reasoning** — when the firmware's logical $6200 reads from a different file offset than the user thinks, the LLM's mental model drifted before it had a chance to be wrong on paper.
- **"Verifying" without actually verifying** — the LLM sometimes claimed it had checked something it had only plausibility-tested. Pre-commit, before the friend's email.



Counterweight slide. These are the real limitations, not strawmen.

The "verifying without verifying" one is worth dwelling on. LLMs will say "I confirmed X" when what they actually did was "I generated text asserting X, with no evidence-gathering step". Detecting this requires either (a) the human asking "show me the evidence", or (b) automated tests that fail when the claim is wrong. Both are valuable.

This is also why the test suite matters. The end-to-end test catches disassembly drift. If a future "improvement" to the spec broke the boot stub disassembly, the e2e test would fail.

The collaboration model

Human

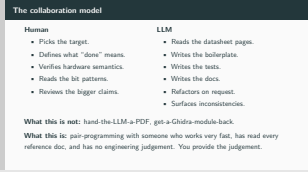
- Picks the target.
- Defines what "done" means.
- Verifies hardware semantics.
- Reads the bit patterns.
- Reviews the bigger claims.

LLM

- Reads the datasheet pages.
- Writes the boilerplate.
- Writes the tests.
- Writes the docs.
- Refactors on request.
- Surfaces inconsistencies.

What this is not: hand-the-LLM-a-PDF, get-a-Ghidra-module-back.

What this is: pair-programming with someone who works very fast, has read every reference doc, and has no engineering judgement. You provide the judgement.



Land the pitch. The collaboration model is the substance of the talk.

The point is: this isn't autonomous coding. It isn't replacement of the engineer. It's a force multiplier on the parts of the work that LLMs are good at, paired with a human doing the parts they're bad at.

For a sceptical audience: this is consistent with what your scepticism says LLMs can't do. The trick is identifying which parts of a project fit the LLM strength profile and which need real engineering judgement.

If you do that mapping correctly, the throughput is genuinely several times what a human alone would deliver. If you don't, you get plausible nonsense fast.

The artefact

The artefact

What's in the repo

github.com/honx/rockwell_l39_ghidra

- Full Ghidra processor module (drop-in)
- SLEIGH spec, ∼600 lines, all R65C19 ISA
- Java loader for banked images
- Three tests, all green, runnable
- Four docs (overview/usage/architecture/open-points)
- 350-line analysis of the ELSA firmware
- Reference firmware excluded by `.gitignore` (not ours to redistribute)

Apache 2.0. Patches welcome. The bug history is in the git log; if you want to see what the LLM got wrong before review, look at commit `20026e9^`.

Land where to find it. Repo is small, self-contained, and includes the wrong-then-right history so anyone can audit.

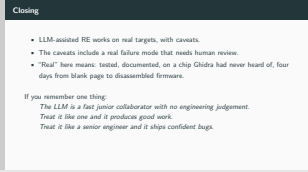
The bug commit message is honest about what changed and why. Reading it is itself an example of the kind of human-AI collaboration the talk is arguing for.

Closing

- LLM-assisted RE works on real targets, with caveats.
- The caveats include a real failure mode that needs human review.
- "Real" here means: tested, documented, on a chip Ghidra had never heard of, four days from blank page to disassembled firmware.

If you remember one thing:

The LLM is a fast junior collaborator with no engineering judgement.
Treat it like one and it produces good work.
Treat it like a senior engineer and it ships confident bugs.



Closing line. Pause, then open Q&A.

If asked: yes, I'm aware this is just one case study. No, I don't have benchmarks. The repo and the test suite are the falsifiable artefact. Anyone who wants to disprove the case study can fork the repo, propose a fix, and see whether the tests still pass.

Questions?

honx@thinventions.de
github.com/honx/rockwell_l39_ghidra

Questions?
honx@thinventions.de
github.com/honx/rockwell_l39_ghidra

Q&A. Anticipated questions:

Q: What model? Sonnet 4.6 / Opus 4.7. The session was Claude Code in auto mode. The work is reproducible with any agentic LLM that has shell + file tools.

Q: How much did you spend? Honestly, low single-digit dollars on inference cost over four sessions. The economics are favourable for this kind of focused work.

Q: Could you have done this without an LLM? Yes, in maybe 4 weeks of part-time evening work. With the LLM it was 4 days. The hard intellectual content (the bug-finding, the BSR encoding correction) was still human work.

Q: What about adversarial use? Out of scope. RE is dual-use; this talk is on the defensive/research side.

Q: Bank-aware analyser, when? PR welcome. It's the obvious next step.